

Deliverable 3 — Web Project (UdL, 2025/26)

Course: Projecte Web **Professors:** Roberto Garcia, David Sarrat **Team:** Carlos Sallán, Maël Dutrey, Eloy Moreno

1. GitHub Public Repository

<https://github.com/CarlosSallan/MTG-EDH-Deckbuilder>

GitHub user `rogargon` was added as a collaborator with read access during Deliverable 1 and still has access.

2. Detail Page Carrying the Semantic Markup

The page chosen for the RDFa markup is the **deck detail page**:

- URL pattern: `/decks/<id>/`
- Template: `templates/blog/deck_detail.html`
- Card sub-template (used 99x per deck): `templates/blog/components/card.html`

Sample URL once the server is running locally: <http://127.0.0.1:8000/decks/1/>

3. Design Considerations

Choice of vocabulary

We use **schema.org** with **RDFa 1.1** syntax (`vocab` , `typeof` , `property` , `resource`), as the consigne explicitly says "semantic markup based on RDFa" and the course tutorial is the "Django Web 3.0 RDFa Tutorial". The validator <https://validator.schema.org> parses RDFa natively.

`vocab="https://schema.org/"` is declared once on the root wrapper of the deck detail block — every `property` / `typeof` underneath inherits it, so we don't have to repeat the namespace.

Mapping our domain to schema.org types

The deck-detail page is annotated as nested schema.org resources:

Domain entity	schema.org type	Justification
Deck	CreativeWork	A deck is an authored, curated arrangement of cards — the closest match in schema.org's generic vocabulary. Same complexity profile as the restaurant-reviews example referenced in the consigne.
Deck.author (User)	Person	Standard mapping. Nested via <code>property="author"</code> <code>typeof="Person"</code> .
Deck.commander, Deck.partner_commander, every DeckCard.card	Product	A Magic card is a physical product (printed, sold, traded). Product exposes the properties we actually have (name, image, category, identifier, sku), which is more than Game or CreativeWork would give us here.

	Each Product is attached to the deck via <code>property="mentions"</code> (CreativeWork → Thing).
--	---

Properties exposed

On the **deck (CreativeWork)**:

- `name` — `deck.name`
- `description` — `deck.description`
- `author` → nested `Person` with `name`
- `url` — emitted via `<link property="url">` using `request.build_absolute_uri`
- `genre` — fixed string "Magic: The Gathering – Commander/EDH deck"
- `dateCreated` — `deck.created_at` (ISO 8601 via `|date:"c"`)
- `dateModified` — `deck.updated_at`
- `resource=<absolute URL>` on the root element to give the CreativeWork a stable IRI
- `mentions` — one nested `Product` per commander, per partner-commander, and per card in the 99-card list

On each **card (Product)**:

- `name` , `image` , `identifier` (Scryfall UUID), `category` (`type_line`), `sku` (`set_code` - `collector_number`)
- For the commander tiles, `name` is rendered as visible text (inside the hover preview). For the 99 cards in the deck list, `name` is emitted via `<meta property="name">` because the existing layout only displays the card image — using `<meta>` lets us declare the name to the parser without changing the visual design.

This gives roughly the same shape and depth as the `PostalAddress` / `restaurant-reviews` examples shown in the course tutorial: one top-level entity with sibling primitive properties, one nested `Person` , and a repeating nested entity (each `Product`) with its own sub-properties.

Why we did not change the data model

The semantic markup is derived entirely from existing fields on `Card` (`scryfall_id` , `type_line` , `set_code` , `collector_number` , `image_url` , `name`) and `Deck` (`created_at` , `updated_at` , plus the relations). The model is **unchanged since Deliverable 2** — no new migration was required for Deliverable 3.

Files touched in this deliverable

- `templates/blog/deck_detail.html` — converted the existing minimal microdata markup (introduced before this deliverable) to full RDFa, added `url` / `genre` / `dateCreated` / `dateModified` on the deck, and added `identifier` / `category` / `sku` on each commander.
- `templates/blog/components/card.html` — added `property="mentions" typeof="Product"` on the wrapper plus `name` / `identifier` / `category` / `sku` / `image` for each of the 99 cards in the deck.
- `docs/delivery3.md` — this document.

Validation procedure

To validate the markup against the Schema.org evaluator:

1. Run the server locally (`uv run python manage.py runserver`).

2. Open a deck-detail page in the browser (e.g. <http://127.0.0.1:8000/decks/1/>) and view source.
3. Paste the rendered HTML into the "Code Snippet" tab of <https://validator.schema.org>.

Expected output: one top-level `CreativeWork` with its primitive properties, one nested `Person` (author), and $1 + (0 \text{ or } 1) + N$ nested `Product` items (commander + optional partner + every card in the deck). No errors should be reported; the validator may emit informational notices for optional `Product` fields not present in our model — those are non-blocking.

4. Grade Division

The three team members contributed equally to this deliverable:

Member	Contribution
Carlos Sallán	33.3%
Maël Dutrey	33.3%
Eloy Moreno	33.3%

5. How to Run

```
# Install dependencies (uv)
uv sync

# Apply migrations (the committed db.sqlite3 is already migrated)
uv run python manage.py migrate

# Start the dev server
uv run python manage.py runserver

# Then open http://127.0.0.1:8000/decks/<id>/
# and paste the rendered HTML into https://validator.schema.org
```